

Pythonデータ分析 ～ yfinanceで株価データを分析しよう ～

この資料について

説明を読んだら、その下にあるコードを**自分で入力**して実行してください。コピペではなく、実際に手を動かして入力することで身につきます。

 やってみよう は自分でコードを書く問題です。  ヒント を参考にしながら挑戦してください。

今日のタイムスケジュール

時間	内容
	Step 0. 準備（インストール・yfinanceとは）
	Step 1. 株価データを取得してみよう
	Step 2. pandasでデータを分析しよう
	Step 3. matplotlibでグラフを描こう
	Step 4. 複数銘柄を比較しよう
	Step 5. 移動平均線でトレンド分析
	Step 6. （発展）scikit-learnで翌日の株価を予測してみよう
	まとめ課題（チャレンジ問題）

Step 0. 準備

0-1. yfinanceとは？

yfinance は、Yahoo!ファイナンスから株価データを無料で取得できるPythonライブラリです。

- Webスクレイピングのように「サイトの構造が変わって読み込めなくなる」心配が少ない
- 数行のコードで世界中の株価・為替・株式指数のデータが取得できる
- pandasの DataFrame 形式でデータが返ってくるので、これまで学んだ知識がそのまま使える

0-2. インストール

まずは以下のコードをセルに入力して実行してください。（Google Colabの場合は既に入っていることもあります、念のため実行しておきましょう）

```
pip install yfinance --quiet

# グラフの日本語文字化け対策（Colab用）
pip install japanize-matplotlib --quiet
```

0-3. ライブラリの読み込み

これから使うライブラリをまとめて読み込みます。

```
import yfinance as yf
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

import japanize_matplotlib
```

エラーが出なければOKです。

Step 1. 株価データを取得してみよう

1-1. ティッカーシンボルとは

株価データを取得するには「ティッカーシンボル（銘柄コード）」が必要です。

会社名	ティッカー
Apple	AAPL

会社名	ティッカー
トヨタ自動車	7203.T
ソニーグループ	6758.T
任天堂	7974.T
Google (Alphabet)	GOOGL
Tesla	TSLA

※日本株は末尾に .T をつけます（東証: Tokyo）。

1-2. 株価データを取得する

Appleの株価データを取得してみましょう。

```
# Tickerオブジェクトを作成
apple = yf.Ticker("AAPL")

# 過去1年分の株価データを取得
df = apple.history(period="1y")

# データの中身を確認
df.head()
```

period には以下のような値が使えます。

指定できる値	意味
"1d"	1日
"5d"	5日
"1mo"	1ヶ月
"6mo"	6ヶ月
"1y"	1年
"5y"	5年
"max"	上場からすべて

1-3. データの中身を確認しよう

これまで学んだpandasの命令を使って、データの様子を確認します。

```
# データの形（行数・列数）
```

```
print(df.shape)
```

```
# 各列の情報（型・欠損値など）
```

```
df.info()
```

```
# 統計量の要約
```

```
df.describe()
```

Open（始値）、High（高値）、Low（安値）、Close（終値）、Volume（出来高）という列があることを確認してください。

やってみよう 1

トヨタ自動車（7203.T）の**過去6ヶ月分**のデータを取得し、先頭5行を表示してください。

```
#
```

💡 ヒント： `yf.Ticker("ティッカー") → .history(period="期間")` の順で書きます。

Step 2. pandasでデータを分析しよう

ここからは、これまで学んだpandasの知識を使ってデータを分析していきます。（以下は `df = Apple` の1年データを使います）

2-1. 終値だけを取り出す

```
close_price = df["Close"]
```

```
close_price.head()
```

2-2. 基本統計量を計算する

```
print("最高値:", close_price.max())
print("最安値:", close_price.min())
print("平均値:", close_price.mean())
print("標準偏差:", close_price.std())
```

2-3. 日ごとの変化率（リターン）を計算する

株価分析でよく使う「前日比の変化率」を計算してみましょう。 `pct_change()` を使うと簡単に計算できます。

```
df["Return"] = df["Close"].pct_change()
df[["Close", "Return"]].head()
```

- Return がプラス → 前日より値上がり
- Return がマイナス → 前日より値下がり

2-4. 一番値上がりした日・値下がりした日

```
# 最も値上がりした日
max_day = df["Return"].idxmax()
print("最も値上がりした日:", max_day, "変化率:", df.loc[max_day, "Return"])
```

```
# 最も値下がりした日
min_day = df["Return"].idxmin()
print("最も値下がりした日:", min_day, "変化率:", df.loc[min_day, "Return"])
```

やってみよう 2

Apple株について、以下を計算してください。

1. Return 列の平均値（平均の日次リターン）
2. 値上がりした日数と値下がりした日数（ `Return > 0` の日数、 `Return < 0` の日数）

```
#
```

💡 ヒント：日数を数えるには `(df["Return"] > 0).sum()` のように条件式と `.sum()` を組み合わせます。

Step 3. matplotlibでグラフを描こう

3-1. 終値の推移を折れ線グラフにする

```
plt.figure(figsize=(10, 5))
plt.plot(df.index, df["Close"])
plt.title("Apple(AAPL) 株価推移（過去1年）")
plt.xlabel("日付")
plt.ylabel("終値 (USD) ")
plt.grid(True)
plt.show()
```

3-2. 出来高（Volume）もあわせて表示する

サブプロットを使って、株価と出来高を上下に並べて表示してみましょう。

```
fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(10, 7), sharex=True)

ax1.plot(df.index, df["Close"], color="blue")
ax1.set_title("Apple(AAPL) 株価と出来高")
ax1.set_ylabel("終値 (USD) ")
ax1.grid(True)

ax2.bar(df.index, df["Volume"], color="gray")
ax2.set_ylabel("出来高")
ax2.set_xlabel("日付")

plt.tight_layout()
plt.show()
```

3-3. 日次リターンのヒストグラム

```
plt.figure(figsize=(8, 5))
plt.hist(df["Return"].dropna(), bins=50, color="skyblue", edgecolor="black")
plt.title("Apple 日次リターンの分布")
plt.xlabel("変化率")
plt.ylabel("日数")
plt.show()
```

正規分布に近い形になっていることが確認できるはずです。

やってみよう 3

Step 1で取得したトヨタ自動車のデータを使って、終値の推移を折れ線グラフにしてください。タイトルや軸ラベルも日本語で入れましょう。

```
#
```

Step 4. 複数銘柄を比較しよう

複数の銘柄を同時に取得して、値動きを比較してみましょう。

4-1. 複数銘柄のデータをまとめて取得する

```
tickers = ["AAPL", "MSFT", "GOOGL", "TSLA"]
data = yf.download(tickers, period="1y")["Close"]
data.head()
```

`yf.download()` を使うと、複数銘柄の終値を一度に取得できます。列が各銘柄になったDataFrameが返ってきます。

4-2. 株価の単位が違うので「正規化」して比較する

Apple(AAPL)とTesla(TSLA)では株価の水準が全然違うので、そのまま比べても値動きの大きさが分かりにくいです。そこで、初日の値を100として、そこからの変化率で比較します。

```
normalized = data / data.iloc[0] * 100

plt.figure(figsize=(10, 6))
for ticker in tickers:
    plt.plot(normalized.index, normalized[ticker], label=ticker)

plt.title("株価パフォーマンス比較（初日=100として正規化）")
plt.xlabel("日付")
plt.ylabel("パフォーマンス")
plt.legend()
plt.grid(True)
plt.show()
```

4-3. 相関係数を調べる

これまで学んだ `corr()` を使って、銘柄同士の値動きの関係を調べます。

```
returns = data.pct_change().dropna()
correlation = returns.corr()
print(correlation)
```

相関係数が1に近いほど「似た値動きをする」、0に近いほど「関係が薄い」、マイナスなら「逆の動きをする」ことを意味します。

やってみよう 4

日本の自動車会社3社（7203.T トヨタ、7267.T ホンダ、7201.T 日産）の株価を6ヶ月分取得し、正規化した上で1つのグラフに重ねて表示してください。

```
#
```


Step 5. 移動平均線でトレンド分析

株価分析の基本テクニックである「移動平均線」を計算してみましょう。移動平均は、これまで学んだpandasの `rolling()` を使います。

5-1. 移動平均線を計算する

```
apple_data = yf.Ticker("AAPL").history(period="1y")

# 25日移動平均と75日移動平均を計算
apple_data["MA25"] = apple_data["Close"].rolling(window=25).mean()
apple_data["MA75"] = apple_data["Close"].rolling(window=75).mean()

apple_data[["Close", "MA25", "MA75"]].tail()
```

5-2. 株価と移動平均線をグラフに重ねる

```
plt.figure(figsize=(10, 6))
plt.plot(apple_data.index, apple_data["Close"], label="終値", alpha=0.6)
plt.plot(apple_data.index, apple_data["MA25"], label="25日移動平均")
plt.plot(apple_data.index, apple_data["MA75"], label="75日移動平均")
plt.title("Apple(AAPL) 株価と移動平均線")
plt.xlabel("日付")
plt.ylabel("株価 (USD) ")
plt.legend()
plt.grid(True)
plt.show()
```

短期線（25日）が長期線（75日）を下から上に抜けるポイントを「ゴールデンクロス」、逆を「デッドクロス」と呼び、売買のサインとして使われることがあります。

やってみよう 5

好きな銘柄を1つ選び、5日移動平均と25日移動平均を計算してグラフに表示してください。

#

Step 6.（発展）scikit-learnで翌日の株価を予測してみよう

これまで学んだscikit-learnの線形回帰を使って、簡単な株価予測にチャレンジします。

※あくまで学習目的の簡易モデルです。実際の投資判断には使えません。

6-1. 特徴量を作る

「何日前からのデータ番号」を特徴量(X)として、「終値」を予測対象(y)とする、非常にシンプルなモデルを作ります。

```
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split

data_ml = apple_data.reset_index()
data_ml["Days"] = np.arange(len(data_ml)) # 0,1,2,...という連番を作る

X = data_ml[["Days"]]
y = data_ml["Close"]

# 訓練データとテストデータに分割（シャッフルしない：時系列データのため）
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, shuffle=False
)
```

6-2. モデルの学習と予測

```
model = LinearRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
```

6-3. 結果をグラフで確認する

```
plt.figure(figsize=(10, 6))
plt.plot(data_ml["Days"], data_ml["Close"], label="実際の株価")
plt.plot(X_test["Days"], y_pred, label="予測値", color="red")
plt.title("線形回帰による株価予測（簡易版）")
plt.xlabel("経過日数")
plt.ylabel("株価（USD）")
plt.legend()
plt.grid(True)
plt.show()
```

6-4. モデルの評価

```
from sklearn.metrics import mean_squared_error, r2_score

print("決定係数(R2):", r2_score(y_test, y_pred))
print("RMSE:", np.sqrt(mean_squared_error(y_test, y_pred)))
```

R2は1に近いほど当てはまりが良いことを意味しますが、株価のような複雑な動きを直線だけで予測するのは限界があることも確認しておきましょう。

まとめ課題（チャレンジ問題）

以下の条件を満たすものを作成してください。

1. 好きな銘柄を**3つ**選び、過去1年分の株価データを取得する
2. 3銘柄の終値の推移を、正規化して1つのグラフに重ねて表示する
3. 3銘柄それぞれの「日次リターンの平均」「日次リターンの標準偏差（値動きの大きさ）」を計算し、表で比較する
4. 3銘柄のうち、どれが最も値動きが安定していたか、コメントを書く（コードの下にMarkdownセルで一言書いてください）

チャレンジ課題：

振り返り

- 別ライブラリの使い方
- `yfinance` を使って、株価データを取得できる
- 取得したデータは `pandas` の `DataFrame`
- `pct_change()` でリターン（変化率）を計算できる
- `rolling()` で移動平均を計算し、トレンド分析ができる
- 複数銘柄を正規化して比較すると、パフォーマンスの違いが分かりやすい
- `scikit-learn`を使えば株価予測にも挑戦できるが、単純なモデルには限界がある